CMPE 322

A3 Feature Enhancement Report

April 7, 2026

Authors:

Jacob Badali
20jasb1@queensu.ca

Zhu Yuan
18zy11@queensu.ca

Chloe Atherton
20cga3@queensu.ca

Will Pritchard
21whp2@queensu.ca

Julian Tiqui
21jtt4@queensu.ca

Connor Rewa
20car8@queensu.ca

Abstract

This report proposes the integration of reusable, persistent custom agent profiles into the Gemini CLI architecture. Currently, execution within the Gemini CLI is stateless, forcing developers to repetitively provide contextual rules and workflow preferences across sessions. The proposed enhancement resolves this by enabling users to define, store, and invoke tailored agent personas—complete with custom execution rules and document attachments—across multiple sessions and projects. To support this stateful feature, we propose expanding the existing layered architecture with a novel Agent Profile Manager subsystem and persistent storage mechanisms. We examine two implementation alternatives: a Local Database Deployment and a Cloud-Managed Model Context Protocol (MCP) Server. The report details the required conceptual and concrete architectural modifications, impacted system components, and newly introduced external interfaces. A Software Architecture Analysis Method (SAAM) evaluation is performed based on key non-functional requirements to determine the optimal realization. Finally, we analyze the effects on maintainability, evolvability, testability, and performance, alongside associated risks and a comprehensive testing strategy.

Contents

1 Introduction **1**

2 Proposed Enhancement Overview **1**

3 Implementation and Architecture **1**
- 3.1 Overall Structure and Components . . . 1
- 3.2 Revisions to Conceptual Architecture . . . 2
- 3.3 Revisions to Concrete Architecture . . . 3
  - 3.3.1 Impact on Subsystems . . . 4
  - 3.3.2 Impacted Directories and Files . . . 4
- 3.4 Global Control Flow . . . 5

4 Use Cases and Sequence Diagrams **5**
- 4.1 Use Case 1: Creating a Custom Agent Profile . . . 5
- 4.2 Use Case 2: Using an Existing Custom Agent . . . 6

5 Realization Alternatives & SAAM Analysis **8**
- 5.1 Realization Alternatives . . . 8
- 5.2 SAAM Analysis . . . 9
  - 5.2.1 Stakeholders . . . 9
  - 5.2.2 Non-functional Requirements . . . 9
  - 5.2.3 Evaluation of Alternatives . . . 9

6 Impact and Risks **10**
- 6.1 General Impact . . . 10
- 6.2 Effects on Non-Functional Requirements (-ilities) . . . 10
- 6.3 Potential Risks (Security, Scalability, and Complexity) . . . 10

7 External Interfaces **11**

8 Testing **12**

9 Data Dictionary **13**

10 Naming Conventions **13**

11 Conclusions **14**

12 Lessons Learnt **14**

13 References **15**

14 AI Collaboration Report **15**

1 Introduction

The Gemini CLI serves as a terminal-first interface for integrating Gemini AI into developer workflows. While its architecture effectively isolates execution and manages tool orchestration, its session-oriented, stateless design limits efficiency for repetitive tasks. Users must repeatedly recreate context, enforce formatting rules, and attach documentation in each session. To address this limitation, this report proposes a major feature enhancement: Reusable Custom Agent Profiles.

The purpose of this report is to analyze the architectural implications of introducing persistent agent profiles to the Gemini CLI. We outline both high-level and low-level architectural changes, focusing on the addition of an Agent Profile Manager subsystem. By shifting the CLI from a stateless assistant to a more workflow-oriented system, this enhancement aims to improve developer productivity and output consistency.

This report is structured as follows. Section 2 describes modifications to the conceptual and concrete architectures. Section 3 outlines changes to external interfaces. Section 4 presents use cases and sequence diagrams. Section 5 evaluates alternative implementations through SAAM analysis. Finally, the report assesses impacts on non-functional requirements, identifies risks, and proposes a testing strategy.

2 Proposed Enhancement Overview

This enhancement introduces reusable custom agent profiles within the Gemini CLI, allowing users to define and invoke personalized AI behaviors across projects. Currently, the CLI operates on a per-session basis, requiring users to repeatedly specify preferences, workflows, and context. This reduces efficiency and consistency, particularly for recurring tasks such as testing, coding, and documentation. The proposed enhancement enables persistent agents that encapsulate personas, formatting rules, and optional supporting documentation such as company standards.

Once created, these agents can be reused across sessions and projects. When invoked, the system automatically applies the agent's persona, rules, and context, ensuring consistent outputs without repeated prompting. For example, a developer could define a testing agent that enforces company protocols or a coding agent that applies preferred formatting styles.

The primary benefit is improved productivity and consistency. By enabling reusable workflows and reducing prompt repetition, users can execute tasks more efficiently while maintaining standardized outputs. This shifts Gemini CLI from a stateless assistant toward a more stateful, workflow-oriented system.

3 Implementation and Architecture

3.1 Overall Structure and Components

The base concrete architecture of the Gemini CLI relies on a four-tier structure containing the Presentation, Application, Service, and Infrastructure layers. This architecture was designed with a focus on maintainability, evolvability, and strict security isolation.

- **Presentation Layer (Terminal Interface):** Responsible for dynamic user input capture

and output rendering via a React-based terminal DOM. It strictly delegates all application logic downwards to prevent coupling presentation from orchestration.

- **Application Layer (Orchestration Engine):** The central controller of the system. It manages conversational turns, asynchronous scheduling, and the performance-critical communication with the external Cloud Model Service.
- **Service Layer (Tool Execution Framework, External Integration Layer, Task Delegation System):** A highly modular layer built fundamentally on a Registry design pattern. It provides capabilities to the LLM (like shell execution or file reading) through dynamic discovery. This registry pattern inherently supports evolvability, as new external tools or agent functionalities can be added without modifying the core orchestrator.
- **Infrastructure Layer (Security & Execution Environment, Configuration System):** Isolated at the bottom of the stack to ensure that all tool executions are validated against strict security policies and user preferences before interacting with the local host.

3.2 Revisions to Conceptual Architecture

To support the proposed reusable custom agents enhancement, the architecture requires targeted expansions rather than a complete overhaul. At a high conceptual level, the system requires the introduction of a new Agent Profile Manager subsystem. Previously, the system treated every session as stateless context only existed within the current run. The new Agent component introduces statefulness by acting as a bridge between user prompts and historical configurations. The Core Orchestration component must be modified to query this new manager before dispatching a request to the LLM so that the persistent persona and its attached documentation can be injected directly into the LLM payload.

Visual architectures of these modifications depend heavily on the realization alternative chosen. As shown in Figure 1, a local or workspace-scoped configuration approach relies on reading locally stored Agent Profiles. Conversely, a cloud-managed approach relies on treating Agent definitions as external dependencies, effectively fetching them from an external Agent MCP Server (see Figure 2).

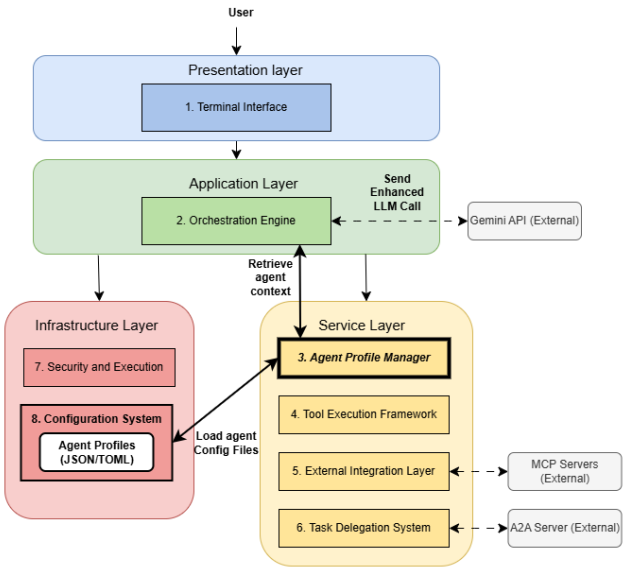


Figure 1: Local Configuration Alternative

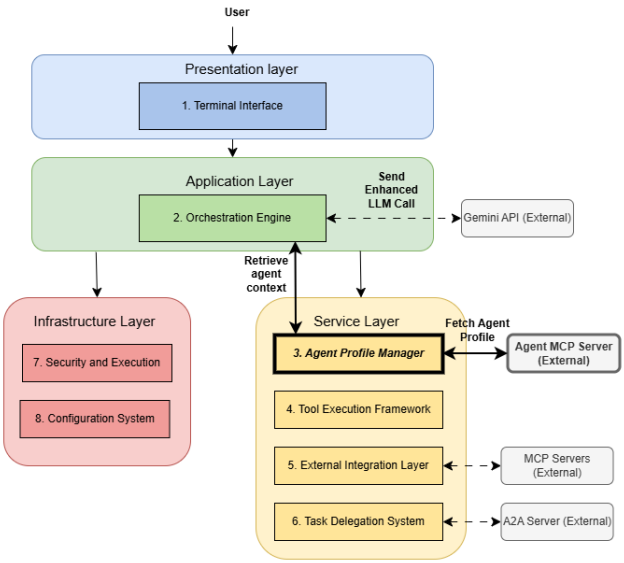


Figure 2: Cloud-Managed MCP Alternative

Because there are two major realization alternatives proposed for this enhancement (Local Deployment vs. Cloud-Managed Deployment), the persistence requirement splits:

- **Alternative 1 (Local Database Deployment):** Introduces a local storage subsystem specifically for agents.
- **Alternative 2 (Cloud-Managed MCP Server):** Introduces a remote reliance, treating Agent definitions as external API dependencies.

3.3 Revisions to Concrete Architecture

At a concrete implementation level, we are migrating from simple file-based configuration loading to database-backed or cloud-backed persistence. This introduces a dual-dependency relationship where the Orchestration Engine strictly depends on the new Agent Profile Manager for context before acting. The Service Layer must be extended to parse attached documents (for testing protocols, coding standards, etc.) and append them dynamically to the context window prior to model invocation.

Figure 3 illustrates the proposed Concrete Architecture and Dependency diagram, identifying all core subsystems, their internal modules, and the flow of dependencies passing between them.

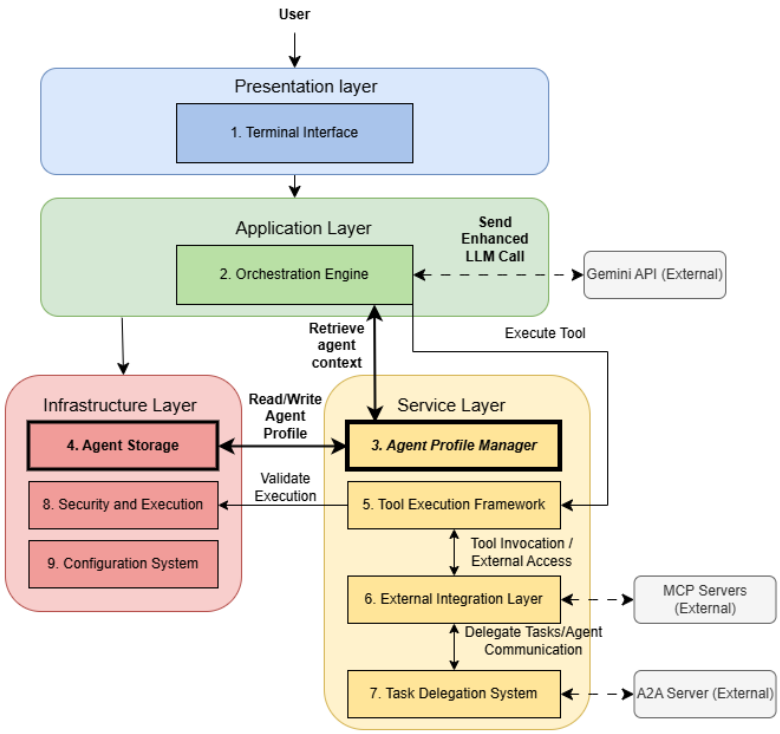


Figure 3: Concrete Architecture and Dependency Diagram

3.3.1 Impact on Subsystems

To implement the custom agent enhancement, the following subsystems will be impacted:

- **Terminal Interface:** Will require a selection menu or flag parser to allow the user to create, edit, or select which agent they wish to run for the current session.
- **Orchestration Engine:** The core execution flow must be modified to accept configurations from the Agent Profile Manager and inject the selected persona's rules and attached documents into the initial system instructions.
- **Tool Execution Framework:** May need to support dynamic restriction rules (e.g., if a "Testing" agent persona restricts the execution of modifying commands like `git push`, the tools must fail gracefully).
- **Agent Profile Manager (New Subsystem):** This new module will handle the business logic of retrieving personas, checking their permissions, and assembling their attached documentation.
- **Agent Database / Cloud Config (New Subsystem):** Provides the persistent storage logic. Will interface directly with local SQLite/JSON storage or a Cloud storage endpoint, managing the CRUD operations for agent profiles.

3.3.2 Impacted Directories and Files

At the source-code level, the following implementation files will be added or modified. The strict layered flow of `packages/cli` relying on `packages/core` will be maintained:

- `packages/cli/src/ui/AppContainer.tsx` & `gemini.tsx`**:** Modified to support new UI states and props for agent selection input passing. This explicitly links back to the Terminal

Interface component in the Concrete Architecture diagram.

- `packages/core/src/core/turn.ts` & `client.ts`**:** Modified to inject the agent properties directly to `geminiChat.ts`. This links directly back to the Orchestration Engine component shown in the architectural dependencies.
- `packages/core/src/agents/agent-profile-manager.ts` **[NEW]:** Creation of the new Service layer component containing the logic for matching user agent requests to the persistent data. This acts as the newly proposed Agent Profile Manager module in the Concrete Architecture diagram.
- `packages/core/src/agents/agent-database.ts` **[NEW]:** Creation of the local persistence logic (if Alternative 1 is chosen). This implements the Agent Database and Persistent Storage subsystems mapped out in the Concrete Architecture.
- `packages/core/src/policy/policy-engine.ts`**:** Modified to read from the Agent Profile Manager to evaluate agent-specific constraints alongside global security constraints. This file realizes the Security and Execution Environment operations noted in the Concrete Architecture.

3.4 Global Control Flow

The global control flow functions on an event-driven, asynchronous model inherent to Node.js. With our enhancement, the control flow is modified as follows: When a user submits a command via the Terminal Interface, control is passed asynchronously to the Orchestration Engine. Before making a network request to the Cloud Model Service, the Orchestration Engine pauses to emit an event to the newly proposed Agent Profile Manager. The manager synchronously (or asynchronously if using the Cloud alternative) retrieves the relevant agent's persona and document embeddings. Once the context is bundled, control resumes within the Orchestration Engine, which dispatches the formatted request to the Gemini API. The overarching architectural style remains a Layered architecture combined with an Event-Driven Publish-Subscribe pattern.

4 Use Cases and Sequence Diagrams

The proposed enhancement enables a persistent, stateful agent workflow. To illustrate the exact mechanisms of these changes, we provide two non-trivial use cases along with sequence diagrams that show how the layers inside the architecture interact.

4.1 Use Case 1: Creating a Custom Agent Profile

This use case describes how a user creates a new reusable custom agent in Gemini CLI. The user provides an agent definition through the Terminal Interface, including the agent's name, persona, behavioral rules, and any attached documentation. The system validates the request, persists the new agent profile, and returns confirmation to the user. This use case demonstrates how the proposed enhancement introduces persistent agent management into the architecture while preserving the existing layered structure.

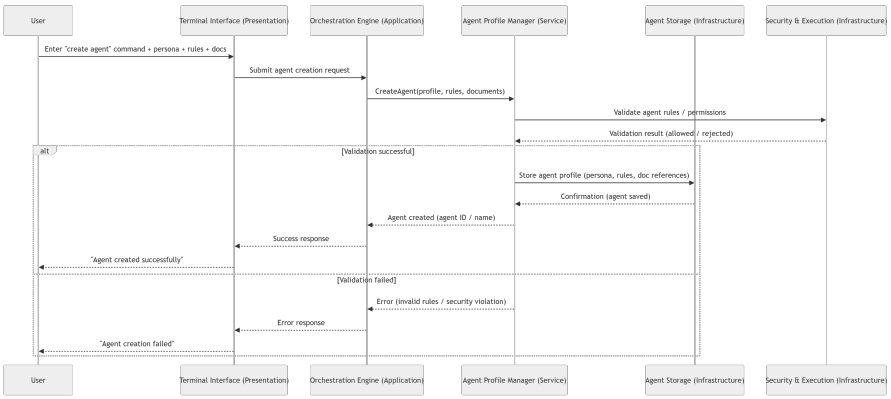


Figure 4: Sequence Diagram: Creating a Custom Agent Profile

Main Flow:

1. The user enters a command through the Terminal Interface to create a new agent, providing the agent's name, persona, rules, and optional supporting documentation.
2. The Terminal Interface captures the request and forwards the agent creation data to the Orchestration Engine.
3. The Orchestration Engine interprets the request as an agent management operation and delegates it to the Agent Profile Manager.
4. The Agent Profile Manager parses the submitted definition and prepares the agent profile for validation and storage.
5. The Agent Profile Manager sends the proposed agent rules and restrictions to the Security and Execution component to verify that the new agent definition does not violate system-level safety or execution constraints.
6. The Security and Execution component evaluates the submitted configuration and returns either approval or rejection.
7. If the configuration is approved, the Agent Profile Manager sends the validated agent profile to Agent Storage for persistence.
8. Agent Storage saves the agent's persona, rules, and references to any attached documentation, then returns a confirmation that the profile was successfully stored.
9. The Agent Profile Manager returns a success response to the Orchestration Engine, indicating that the new agent has been created.
10. The Orchestration Engine forwards the result to the Terminal Interface, which displays a confirmation message to the user.

Alternate Flow: If the submitted persona, rules, or attached configuration violate safety constraints, the Security and Execution component rejects the request. In that case, the Agent Profile Manager returns an error to the Orchestration Engine, which forwards the failure message to the Terminal Interface so the user can revise the agent definition and try again.

4.2 Use Case 2: Using an Existing Custom Agent

This use case describes how a user invokes a previously created custom agent (e.g., a "Documentation Agent") to perform a task using its predefined persona, rules, and attached documentation. The system retrieves the agent's stored profile, injects its context into the execution flow, option-

ally gathers relevant project information, validates execution constraints, and generates output that follows the predefined structure.

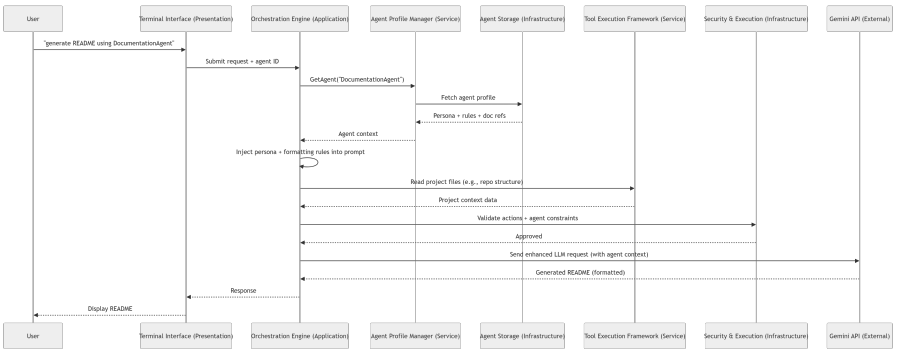


Figure 5: Sequence Diagram: Using a Custom Agent Profile

Main Flow:

1. The user enters a command through the Terminal Interface to generate a README file using the "Documentation Agent," along with a prompt such as "generate a README for this project."
2. The Terminal Interface captures the request and forwards it to the Orchestration Engine, including the selected agent identifier.
3. The Orchestration Engine sends a request to the Agent Profile Manager to retrieve the corresponding agent profile configuration.
4. The Agent Profile Manager queries Agent Storage and retrieves the stored definition of the Documentation Agent, including its persona, formatting rules, and attached documentation references.
5. The assembled agent context is injected by the Orchestration Engine into the user request to construct an enhanced LLM prompt.
6. The Orchestration Engine may request additional context from the Tool Execution Framework, such as reading relevant project files from the local development environment.
7. The Orchestration Engine sends the combined request and context to the Security and Execution component to validate that all actions comply with system policies.
8. Once validated, the Orchestration Engine sends the enhanced request, including agent context and project information, to the Gemini API.
9. The Gemini API processes the request and returns generated content that strictly follows the Documentation Agent's predefined structure and formatting rules.
10. The Orchestration Engine receives the response and forwards it to the Terminal Interface to display.

Alternate Flow: If the selected agent cannot be retrieved from storage, the Agent Profile Manager returns an error to the Orchestration Engine, which notifies the user. Additionally, if the request violates system policies or agent-defined constraints, the Security and Execution component rejects the operation, and the system returns an error.

5 Realization Alternatives & SAAM Analysis

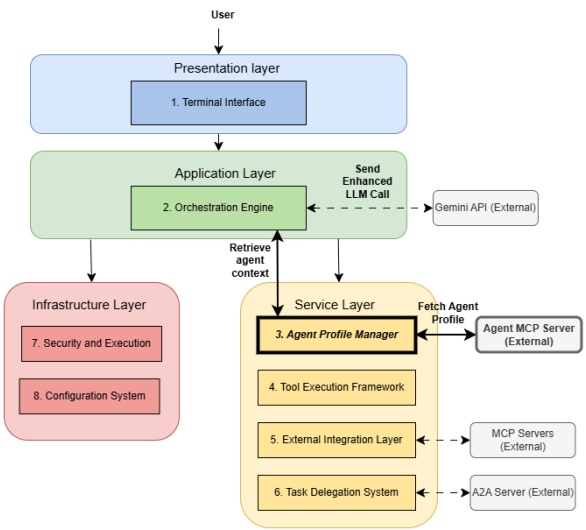
5.1 Realization Alternatives

There are three main ways in which the proposed reusable custom agent enhancement can be realized in the Gemini CLI architecture. In all cases, the enhancement introduces a new Agent Profile Manager component that acts as an intermediary between the user request and the Orchestration Engine. The purpose of this component is to retrieve the selected agent profile, assemble its stored persona and attached documentation, and provide this persistent context before the request is sent to the model. The difference between the alternatives lies mainly in where the agent profiles are stored and how they are retrieved by the system.

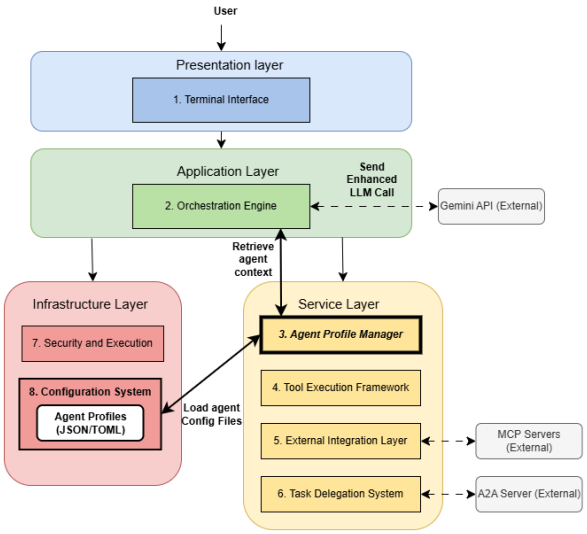
Alternative 1 (Local Database Deployment): Agent profiles are stored locally on the user's machine using a local database or structured storage. The Agent Profile Manager retrieves the profile before passing its context to the Orchestration Engine. This keeps the enhancement within Gemini CLI's existing local execution environment and aligns perfectly with its focus on local capability, latency reduction, and strict security isolation. This architecture has been described in Figure 3, above.

Alternative 2 (Cloud-Managed MCP Server): Agent profiles are stored remotely and accessed through a cloud-based service exposed via an MCP interface. The Agent Profile Manager retrieves profiles from an external endpoint. This makes the system highly distributed and relies heavily on the External Integration Layer, enabling cross-device sharing but pushing persistence responsibilities outside the local host. This architecture is outlined in Figure 6a below.

Alternative 3 (Workspace-Scoped Configuration File): Agent profiles are stored as structured configuration files (JSON/Markdown) inside a user's global directory or project workspace. The configuration is loaded natively through the Configuration System. Like the database approach, this is local and low-latency, but it provides a lighter-weight mechanism that is easily version-controlled and shared via repositories. This is shown in Figure 6b below.



(a) MCP Cloud Based Agent Store



(b) Configuration Based Agent Store

Figure 6: Architectural Alternatives Considered

5.2 SAAM Analysis

5.2.1 Stakeholders

End user (developer): Concerned primarily with latency and reliability. Because this feature targets eliminating repetitive prompting, UX responsiveness is critical. **Organizations/Teams:** Concerned with enforceability, policy management, and the ability to share safe constraints across multiple users. **Gemini CLI maintainer:** Responsible for ensuring this enhancement does not violate the strict concrete separation of concerns across the project's monorepo layers. **Security/Compliance:** Concerned with protecting sensitive data. Expanding an agent's memory indefinitely with proprietary documents introduces massive data-leak risks that must be sandboxed.

5.2.2 Non-functional Requirements

Performance & Usability: Retrieving agent definitions must not perceptibly delay execution. **Consistency & Manageability:** If agents cannot easily be deployed organization-wide, core workflow benefits are minimized. **Modifiability & Maintainability:** Any approach must keep Orchestration logic distinctly independent from the storage mechanisms. **Security & Privacy:** Persisting context profiles mandates strict sandboxing to prevent the Cloud model from improperly harvesting local documents.

5.2.3 Evaluation of Alternatives

Local Database Deployment: The Local Database Deployment performs strongly in terms of performance and reliability. Since agent profiles are stored locally, they can be retrieved without network dependency, reducing latency and enabling a responsive and stable experience, even in offline environments. It also provides strong security and privacy benefits, as sensitive data remains on the user's machine. However, it is weaker in terms of organizational manageability and consistency, as profiles are not centralized and may require manual synchronization across users. From a maintainability perspective, it introduces minimal complexity and aligns well with the existing layered architecture, making it easier to implement, test, and evolve.

Cloud-Managed MCP Server: This approach performs well in terms of organizational consistency and centralized management, as profiles can be shared across users and environments. However, it introduces performance and reliability drawbacks due to network dependency, including latency and potential service outages. It also increases reliance on external integrations and adds operational complexity. From a security standpoint, storing and transmitting profiles remotely increases the attack surface and requires stronger access controls. Overall, while it improves sharing, it comes at the cost of performance, security risk, and increased architectural complexity.

Workspace-Scoped Configuration File: This alternative performs well in performance and reliability, as profiles are stored locally and retrieved without network dependency. It also improves manageability for small teams, as profiles can be easily reviewed, versioned, and shared as files. However, it is weaker in terms of security and structured persistence, as files may be duplicated or exposed and lack controlled storage mechanisms. From a maintainability perspective, it integrates well with the existing Configuration System and is simple to implement, but may not scale effectively as the number of profiles grows.

SAAM Conclusion: The **Local Database Deployment** is the strongest overall solution, providing the best balance of performance, security, reliability, and maintainability while aligning with the CLI's architecture. The configuration-based approach is a useful lightweight alternative, while the cloud-managed option introduces greater complexity, external dependencies, and security concerns.

6 Impact and Risks

6.1 General Impact

This modification will make the application more flexible due to the ability of supporting a greater interaction flow that the current system's architecture cannot. In particular, from an abstract point of view, the main effect here will be the fact that now, instead of the request being processed only within one thread or a single chain of events, the system should support more actions like coordinating activities across a stored database, managing states, and applying constraints.

One of the major effects of this enhancement is the increase in architectural complexity. Namely, previously stateless components must now handle context synchronization. The Orchestration Engine will need to perform more complex tasks such as handling coordination states, communicating with the new Agent Profile Manager, and orchestrating the caching of external document embeddings.

6.2 Effects on Non-Functional Requirements (-ilities)

Maintainability and Evolvability: The enhancement improves maintainability by introducing agent definitions that centralize repeated logic and reduce duplication. However, it also adds complexity through new components for creation, storage, retrieval, and context injection. Maintaining clear interfaces and separation of concerns is essential to avoid tight coupling across the Orchestration Engine, Configuration System, and Tool Execution Framework. From an evolvability perspective, the enhancement is beneficial, enabling modular extensions without modifying core logic and supporting future capabilities such as shared agents or multi-agent coordination.

Testability: The enhancement improves testability by introducing reusable, structured personas that enable consistent validation of LLM behavior. However, the addition of persistent state increases system complexity and the number of interactions that must be tested. This requires comprehensive unit and integration testing, including validation of the Agent Profile Manager, storage mechanisms, and rule enforcement.

Performance: Introducing persistent state and larger context payloads increases overhead and may reduce responsiveness. While the architecture remains efficient, performance impacts arise from context injection and retrieval. These effects, particularly under large-scale usage, are further addressed as system risks in Section 6.3.

6.3 Potential Risks (Security, Scalability, and Complexity)

As explicitly noted during the design phase, allowing users to upload and attach additional documentation introduces two primary risks: **Security/Privacy** and **Scalability**.

- **Security and Privacy Risks:** A central concern is that custom agents may ingest and indefinitely store highly sensitive information, such as proprietary company testing protocols, internal coding standards, or confidential API keys. If the agent profiles are improperly stored, or if a Cloud-Managed distribution architecture is adopted, this proprietary data could be inadvertently exposed or leaked across organizational boundaries. Strict sandboxing, local-first architectures, and role-based validation within the Security & Execution Environment are critical to mitigating this.
- **Scalability and Performance Risks:** Introducing stateful agent behaviors requires retrieving definitions and potentially loading vast swaths of associated documentation during each invocation. Supplying this context to the LLM increases token limits and context-window strain. Without efficient document retrieval (such as RAG or semantic chunking) and strategic caching, the system may suffer severe latency degradation, defeating the responsive expectations of a CLI application.
- **Usability and Misuse Risks:** If users misconfigure agent rules (e.g., conflicting security limits), it could yield unpredictable or continuously failing executions. The CLI must guarantee predictability and transparency regarding which "Persona" is active to prevent user confusion.

In summary, while the enhancement adds profound workflow power to the Gemini CLI, its adoption must be architecturally isolated to combat complexity and stringently evaluated to defend user privacy and performance constraints.

7 External Interfaces

This enhancement needs the system to engage with various resources and entities that lie beyond the confines of the system architecture. This is important since the enhanced reusable custom agent profile would be more than just an internal component and relies on information that must enter into the system and persist semi-permanently. It is also important for these information assets to be recalled and used during runtime. Thus, the enhancements would affect the internal structure of the system as well as the external interfaces with the user, models, local resources, and configurations.

User Command Interface: The primary interface through which the system and the end-user interact. End-users use this boundary to create, edit, select, invoke, and manage reusable agent profiles. The data transferred includes agent definitions, behavioral rules, references, and execution requests, ensuring outputs reflect the stored agent persona. In the concrete architecture, this involves the Terminal Interface alongside command parsers responsible for agent-related actions.

Cloud Model Service Interface: This links the Gemini CLI to the external model service responsible for providing inference and generating responses. With this proposed enhancement, its payload structure is significantly expanded. In addition to passing the direct request and current session context, the Orchestration Engine must seamlessly inject the selected agent profile's persona, rules, and any attached documentation summaries directly into the model's context window.

Local Development Environment Interface: This interface mediates between the system and the local file system. It becomes increasingly significant because agents can depend on project-specific standards, local test files, and code conventions attached to their profiles. The Tool

Execution Framework will facilitate this interface, allowing agents to dynamically read workspace artifacts and merge the local context with their predefined computational behavior.

Persistent Storage Interface: This newly introduced storage boundary is crucial for the reusable agent functionality. Because agents are preserved across sessions, this persistence mechanism must securely manage saving, retrieving, modifying, and removing profiles. Specifically, data such as agent metadata, formatting instructions, and document attachments are routed through the Agent Profile Manager to support reliable CRUD operations.

Tool Invocation Interface: This interface helps the system interact with execution mechanisms extending beyond plain-text generation. Agent profiles can restrict or manipulate how tools are executed (e.g. blocking destructive commands or changing build flows). Thus, the Tool Execution Framework must become strictly "agent-aware," tightly coupling with the Orchestration Engine to enforce agent-specific execution constraints.

Configuration Interface: This describes how the system interacts with external configuration sources. The enhancement adds another layer of artifacts that interact with run-time settings and environmental factors. Data exchanged includes default agent visibility, storage paths, and permission boundaries, ensuring that agents respect the global limitations managed by the Configuration System.

Documentation Attachment Interface: An optional but powerful interface that regulates the interaction between external documents and their integration with agents. The data transmitted involves file pointers and the parsed content of documents (such as organization rules). This interface acts as the bridge between persistent agent definitions and external knowledge sources, handled via the intersection of file processing and agent management subsystems.

8 Testing

To ensure the custom agent enhancement integrates reliably with the existing Gemini CLI architecture, testing focuses on the correctness of the Agent Profile Manager, its persistence mechanism, and its interaction with existing orchestration and execution components. Testing is conducted at three levels: unit testing, integration testing, and security and scalability validation.

Unit Testing: Unit tests validate the functionality of the Agent Profile Manager and its storage mechanism. This includes ensuring agent personas, rules, and attached documents are correctly parsed and assembled into valid context. The storage component is tested using standard CRUD operations to ensure safe and accurate persistence.

Regression tests are also applied to the Orchestration Engine to ensure that baseline stateless behavior remains unchanged when no agent is selected.

Integration Testing: Integration testing evaluates interactions between the enhancement and existing subsystems. This includes verifying that agent-defined constraints are correctly propagated to the Security & Execution environment through the Tool Execution Framework, preventing unauthorized actions.

Tests also validate that the Orchestration Engine correctly injects agent context into model requests without breaking prompt structure or exceeding context limits. Additionally, end-to-end

flows from the Terminal Interface to the Agent Profile Manager are tested to ensure reliable handling of agent creation, selection, and document attachment.

Security and Scalability Validation: Security testing ensures that agent data is protected through strict access control and cannot be exposed across sessions or leaked to external services. Policy regression tests confirm that agents cannot bypass sandbox restrictions.

Scalability testing evaluates system performance under large agent profiles and complex contexts, ensuring the Orchestration Engine handles context injection without excessive latency. For cloud-based alternatives, network latency and failure scenarios are simulated to ensure graceful degradation and continued usability.

9 Data Dictionary

- **Terminal Interface:** The user-facing boundary built with React/Ink.
- **Orchestration Engine:** The central controller managing session state and LLM routing.
- **Tool Execution Framework / Registry:** Subsystems that dynamically discover and execute local tools.
- **External Integration Layer:** Connects the CLI to external tools via MCP.
- **Task Delegation System:** Manages multi-step workflows.
- **Security & Execution Environment / Policy Engine:** Infrastructure components enforcing safety constraints and validating tool operations.
- **Configuration System:** Manages runtime configuration.
- **Gemini API:** The cloud-based model service used for reasoning.
- **MCP / A2A:** Protocols enabling external tool exposure and distributed execution.
- **Understand:** A static analysis tool used to extract the concrete architecture.

10 Naming Conventions

- **Layer Naming:** The architecture is organized into four layers: Presentation, Application, Service, and Infrastructure. Layer names are written in Title Case and reflect logical system separation.
- **Component Naming:** Components are named based on functional responsibility (e.g., Orchestration Engine, Tool Execution Framework). Names are conceptual and do not directly correspond to individual files.
- **File Naming:** Source files follow kebab-case or camelCase (e.g., tool-registry.ts, read-file.ts). Test files follow the pattern: `<name>.test.ts`.
- **Directory Structure:** The system is organized as a monorepo under `packages/`, with primary subsystems: `packages/cli` and `packages/core`. Functional groupings exist within `core/src` (e.g., tools, agents, policy).
- **Technology Indicators:** `.ts`: TypeScript source files and `.tsx`: TypeScript files with React components (used in CLI UI).
- **Abbreviations:** CLI: Command Line Interface. MCP: Model Context Protocol. A2A: Agent-to-Agent communication. API: Application Programming Interface.

11 Conclusions

This report evaluated the feasibility of introducing reusable custom agent profiles as an architectural enhancement to the Gemini CLI. Through SAAM analysis, we determined that a Local Database Deployment provides the best balance of performance, security, and maintainability. The system's modular architecture—particularly the separation between the Orchestration Engine, Tool Execution Framework, and Security & Execution Environment—enables this enhancement to be integrated without major structural changes. By introducing an Agent Profile Manager as an intermediary, agent context can be safely retrieved and injected prior to model invocation, preserving separation of concerns. The existing architecture supports persistent state management and controlled execution, allowing reusable workflows to be applied consistently without modifying core logic. Overall, the Gemini CLI is well-suited to evolve from a stateless assistant into a stateful, workflow-oriented system while maintaining extensibility, safety, and modularity.

12 Lessons Learnt

Through analyzing the Gemini CLI architecture and evaluating the proposed agent profile enhancement, several key insights emerged:

- **Architecture Enables or Limits Features:** The feasibility of introducing reusable agent profiles is directly dependent on the system's modular design. The separation between orchestration, execution, and configuration enables persistent agent behavior to be integrated without requiring major structural changes.
- **Statefulness Introduces New Complexity:** Transitioning from a stateless system to one supporting persistent agent profiles introduces challenges related to storage, consistency, and context management. This highlighted the importance of the Configuration System and orchestration layer in managing persistent state.
- **Security Must Scale with Capability:** Allowing agents to automatically apply behaviors and execute workflows increases the risk of unintended actions. The existence of a dedicated Security & Execution Environment ensures that these risks can be controlled through policy enforcement and validation.
- **Concrete Architecture Reveals Integration Points:** While the conceptual architecture suggested where the enhancement might fit, the concrete analysis using Understand revealed the exact components—such as the Orchestration Engine and Tool Execution Framework—where agent profiles would realistically integrate.
- **AI Systems Require Structured Control Layers:** The system demonstrates that AI-driven functionality is not simply model output, but requires structured pipelines involving orchestration, validation, and execution. This becomes even more critical when introducing reusable and automated behaviors through agent profiles.
- **Tooling is Critical for Architectural Validation:** Static analysis tools were essential in confirming that the system could support the proposed enhancement without violating architectural boundaries or introducing tight coupling.

13 References

1. Group Report, "A1 Report: Gemini CLI Conceptual Architecture," course project submission, 2026.
2. Group Report, "A2 Report: Gemini CLI Concrete Architecture," course project submission, 2026.
3. Google Gemini CLI source,

14 AI Collaboration Report

AI Member Profile and Selection Process: For this deliverable, our team used two AI tools: OpenAI GPT-4 (via ChatGPT Plus) and Anthropic Claude (via Antigravity IDE). These tools were selected based on task suitability. GPT-4 was utilized for structured writing, diagram formatting, and iterative refinement due to its ability to follow detailed instructions. Claude was used within the IDE for repository exploration and summarization, leveraging its ability to work with longer code context.

Tasks Assigned to the AI Teammate: AI was assigned tasks that benefited from structured generation, such as drafting initial architectural explanations, improving writing clarity, generating Mermaid diagram syntax, and simplifying complex terminology. AI was *not* used to define subsystem boundaries, perform reflexion analysis, interpret dependency graphs, or make architectural decisions. All core analysis was conducted strictly by human engineers using the repository and the Understand modeling tool.

Interaction Protocol and Prompting Strategy: We used a semi-centralized workflow where members interacted with the AI and reviewed outputs collectively before inclusion. Early prompts were general and vague, prompting us to adopt rigid, constraint-based strategies. We improved quality by supplying high-fidelity context (such as directory structures or screenshots) and imposing explicit constraints, such as demanding the AI trace specific orchestration layers (packages/core/src/core/turn.ts) without inventing nonexistent variables.

Validation and Quality Control Procedures: To eliminate architectural hallucination errors, we applied a strict validation process. Every algorithmic claim was verified against the Gemini CLI repository, and structural flows were cross-checked using Understand dependency graphs. All diagrams and text were manually reviewed to ensure correct tool execution flows and systemic accuracy. AI outputs were treated purely as friction-reducing drafts, never as authoritative sources.

Quantitative Contribution to Final Deliverable: AI contributed approximately 30–35% of the final deliverable's surface generation, heavily biased toward syntax and drafting ($\sim$15% stylistic refinement, $\sim$8% diagram syntax, $\sim$12% terminology clarification). Conversely, human engineering drove 100% of the actual architectural deductions, feasibility scoring, and risk evaluations.

Reflection on Human–AI Team Dynamics: While AI massively improved our efficiency in drafting and formatting, it required significant compensatory effort in prompt refinement and manual physical validation. The collaboration highlighted that AI provides a powerful acceleration layer for drafting, but inherently lacks the localized critical logic necessary to independently judge structural trade-offs regarding safety and distributed execution scaling across our unique CLI architecture.